

Agile Development II: XP, User Stories and Kanban

The engineering half of agile, the unit of work a sprint consumes, the discipline of flow — and the line above which SANTE-BASSAM must not be run agile at all.

DATE**Friday, 25 September 2026****MEETING TIME****10:00 - 11:20 AM****ROOM FORMAT****Lecture + Kanban exercise**

06

3 concepts worked on SANTE-BASSAM **1** assigned to you (Kanban)

9 XP practices made concrete **8** stories in the flow log **6** figures

0

Where Session 5 left us

Scrum gave the team a date on which the clerk touches the software. It said nothing about how to write the code, what a backlog item looks like, or what to do when the sprint boundary stops helping. Today supplies those three things — and the line above which none of them apply.

SESSION 5

Manifesto · Scrum

Cheap feedback justifies the trade; Scrum schedules it. Sprint 1: S-04, S-05, S-06 — 16 points.

TODAY

XP

Nine interlocking engineering practices — TDD and CI make the rest safe.

TODAY

User stories & INVEST

The unit of work a sprint consumes; the "so that" clause decides priority.

TODAY · ASSIGNED

Kanban

Columns, WIP limits, Little's Law. Defined here; worked by you in the exercise.

1 Learning outcomes

L0 6.1 Name six or more XP practices, explain why they interlock, and show what each means for a four-student team.

L0 6.2 Write a well-formed user story with Given/When/Then acceptance criteria and evaluate it against INVEST.

L0 6.3 Build a Kanban board with WIP limits, compute cycle time and throughput from a flow log, and find the bottleneck.

L0 6.4 Identify which parts of SANTE-BASSAM are a poor fit for agile, and say what to do about them.



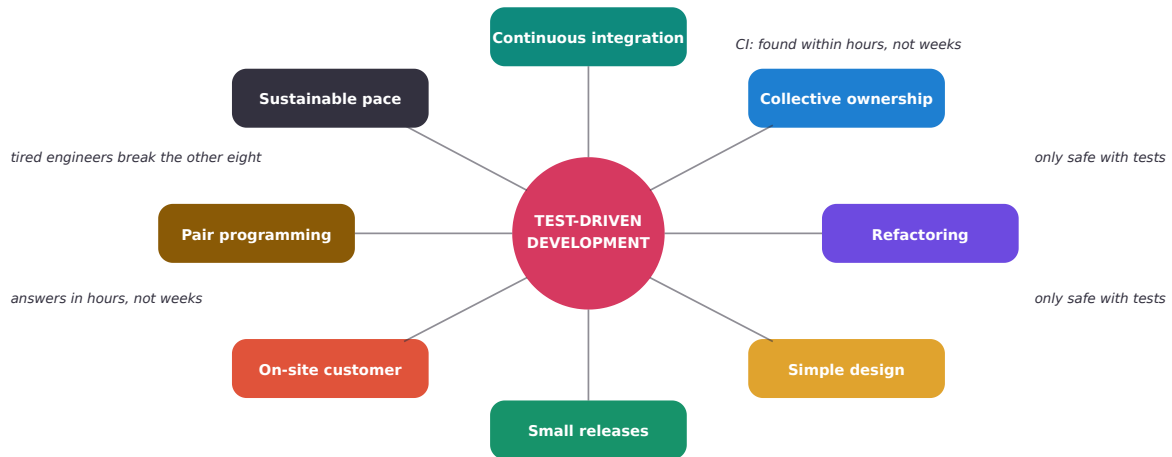
How this session is built. XP, user stories and the limits of agile are worked in full. **Kanban is defined but not worked** — it is the in-class exercise, with a real flow log from the project.

3.1 Extreme Programming: the engineering half

One rule: XP's practices interlock — each is uncomfortable alone and safe only in combination. Collective ownership without tests is chaos; refactoring

without tests is vandalism; simple design without refactoring can't absorb change.

FIG. 1 · NINE PRACTICES AND WHAT MAKES EACH ONE SAFE



Teams that adopt XP selectively — usually pairing (sociable) and skipping TDD (uncomfortable) — frequently make things worse, and then blame the practice.

APPLIED TO SANTE-BASSAM

Nine practices, each made concrete for this team, this semester

TDD

UT-02 is written first

`rank(("Kofi", "Modeste", 2019), ...)`
must return Modeste first with score ≥ 90 . It fails. Then `rank()` is written until it passes. The test is the spec.

CONTINUOUS INTEGRATION

GitHub Actions on every push

Whole suite runs; merge to `main` blocked on red. Set up in Sprint 0. Nobody merges red.

PAIR PROGRAMMING

The one who heard the clerk sits with the one who didn't

The Friday visitors pair on Monday's changes. Pairs rotate each sprint.

COLLECTIVE OWNERSHIP

No module has an owner

Whoever wrote SyncQueue in Sprint 0 may be on the tablet UI in Sprint 3. Anyone fixes a bug wherever it is.

REFACTORING

rank() → similarity() + rank()

When S-15 needs the similarity score, the split happens under existing UT-02: behaviour unchanged, structure improved.

SIMPLE DESIGN

No user roles in Sprint 1

One clerk, one laptop. Roles arrive in Sprint 3 when the nurse needs a screen the clerk shouldn't see — not before.

SMALL RELEASES

Every review Friday, installed at the post

Even if it adds one screen. The clerk keeps using the previous one meanwhile.

ON-SITE CUSTOMER

Fortnightly review + a WhatsApp channel

The PO may ask the nurse one question a day, answered within 24 h. The instructor answers when she cannot.

SUSTAINABLE PACE

16 points, not 30

That is what four students carrying four other courses completed in Sprint 0. No all-nighters before reviews; the review shows what is done.



The failure to avoid. Pairing without TDD = two people confidently writing untested code. On SANTE-BASSAM the untested code is the sync path, and the consequence is a lost health record. Adopt all nine, or know exactly what you are giving up.

3.2 User stories and INVEST

A story is **a promise of a conversation, not a specification**; the acceptance criteria are what the conversation produced. The clause students omit — *so that* — is the one that carries the value: without it you cannot prioritise, and you cannot tell when a cheaper solution would serve.

FIG. 2 · ANATOMY OF A STORY, AND THE SIX-LETTER TEST

As a (role) , I want (capability) , so that (benefit)
 ← the clause that decides priority

I Independent needs nothing unfinished	N Negotiable details came from a talk	V Valuable benefit names real work	E Estimable you can put points on it	S Small fits in one sprint	T Testable Given / When / Then
--	---	--	--	--	--

APPLIED TO SANTE-BASSAM

Two stories worked in full, and one bad one repaired

S-07

As a CHW, I want to register a new patient while the tablet is offline, **so that** I can enrol families during village outreach with no network.

SR-2.2

- AC1** Given no connectivity, when I save a registration, then it is stored locally, marked *pending*, and I see a confirmation with the local patient id.
- AC2** Given pending records, when connectivity returns for ≥ 10 s, then all upload in creation order and each is marked *synced* only after the server acknowledges.
- AC3** Given a record fails three times, then it is flagged for supervisor review and stays readable on the device.

I needs S-01/02 – done

N "10 s", "three times" came from a conversation

V outreach is the CHW's actual job

E 5 pts – Sprint 0 built the store

S fits a sprint

T AC1-3 = ST-03, ST-04 almost word for word: traceability is free

S-12

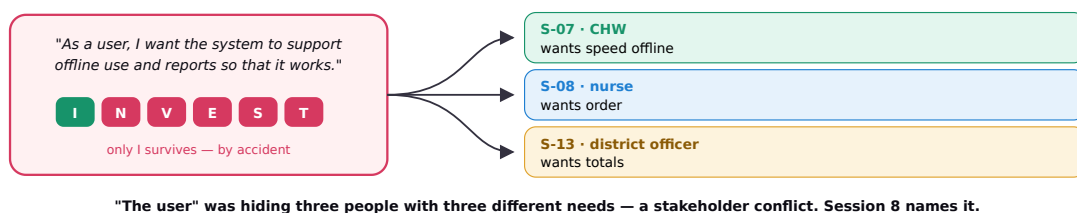
As the nurse, I want a list of patients due for follow-up this week, **so that** I can retire the private notebook I keep because the register cannot do this.

UR-5 (new)

- AC1 Given visits with a follow-up date, when I open the list on a Monday, then I see every patient due this week, sorted by village.
- AC2 Given a listed patient attends, when the visit is recorded, then they leave the list.
- AC3 Given a listed patient has no phone, then the list shows the CHW for their village instead.

The benefit clause here is doing real work: it records why S-12 outranks card printing (S-16). **A private notebook is the strongest evidence a requirements engineer ever finds** — a workaround built by the person who lives with the gap. Session 8 returns to this.

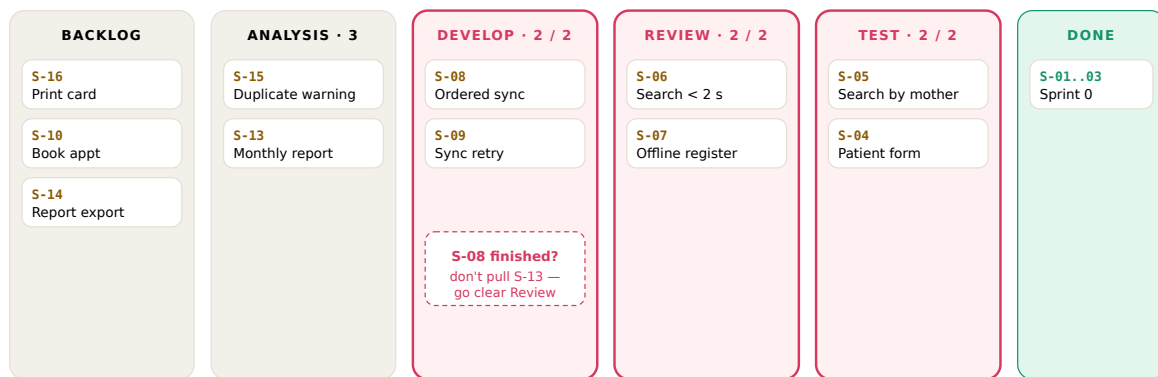
FIG. 3 · A BAD STORY, AND WHAT SPLITTING IT REVEALED



3.3 Kanban and the discipline of flow

One rule: no timebox — work is visualised in columns, each with a *work-in-progress limit*, and new work may not enter a full column. The team's job is to **move work through, not to start work**.

FIG. 4 · A SANTE-BASSAM BOARD MID-SPRINT 2 — READ IT BEFORE YOU READ THE CAPTION



Little's Law: average WIP = throughput × average cycle time → halve WIP, halve cycle time, nobody works harder

Develop, Review and Test are all full, so nothing can move. A developer who finishes S-08 *may not* start S-13: the correct move is to go and help clear Review. WIP limits convert individual busyness into system throughput. **Cycle time** = elapsed time from entering the first active column to Done; **throughput** = items completed per unit time.

ASSIGNED — NOT WORKED

Kanban for the SANTE-BASSAM team is today's in-class exercise

Build the board for your own workflow, set WIP limits, and compute cycle time and throughput from a real flow log of eight stories (section 4). Four worked translations in this document show the standard.

3.4 Where agile does not fit

No accessible customer

The feedback loop is broken; the PO becomes a guess in a chair.

Feedback is genuinely expensive

Spacecraft, implanted devices, systems that cannot be partially deployed.

Regulatory evidence dominates

Certification needs traceable specifications; agile teams can produce them, but must plan to.

Fixed-price, fixed-scope contract

The commercial structure contradicts "responding to change".

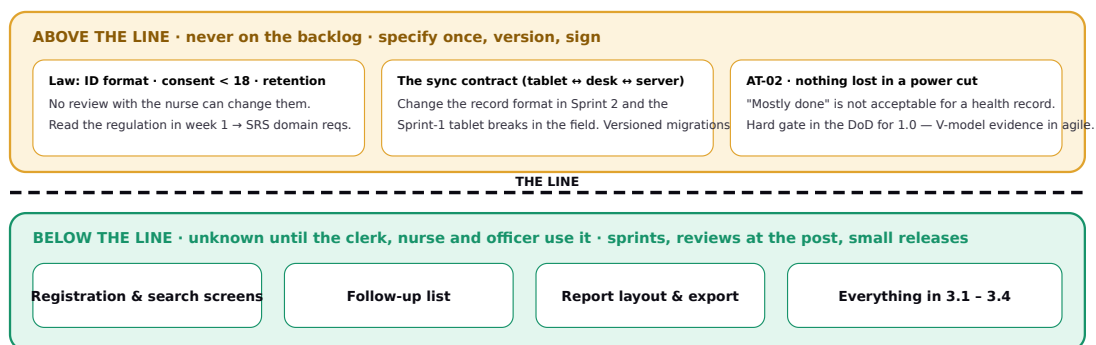
Fixed external interfaces

Iterate behind the interface, never the interface itself.

APPLIED TO SANTE-BASSAM

The line your Project Plan draws

FIG. 5 · ABOVE THE LINE: SPECIFIED ONCE. BELOW THE LINE: DELIVERED IN SPRINTS.



The position, restated. Your Project Plan does not choose between plan-driven and agile. It draws a line. Knowing where to draw it is the engineering judgement this unit is about.

4

In-class exercise · Kanban for your team

Groups of four, 20 minutes. Kanban was defined in 3.5 and not worked — do it now.



Design the board

— columns for *your* workflow (do you need an "At the post" column?). A WIP limit for every active column, one sentence justifying each number for a team of four.



Compute from the log

— cycle time per story in working days, the average, throughput in stories/week over four weeks. Apply Little's Law: what WIP was the team actually carrying?



Find the bottleneck

— six of eight stories spent more days in one column than in all others combined. Which? One WIP change + one non-WIP change (practice, role, rota), with the estimated effect.



Argue the trade

— three sentences: should the team switch to Kanban for Sprints 3-5, given the review with the nurse is what makes the sprint boundary useful?

STORY	ENTERED DEVELOP	LEFT DEVELOP	LEFT REVIEW	DONE (TEST PASSED)
S-04	Mon wk5	Wed wk5	Mon wk6	Tue wk6
S-05	Mon wk5	Fri wk5	Thu wk6	Fri wk6
S-06	Thu wk5	Fri wk5	Thu wk6	Fri wk6
S-07	Mon wk7	Wed wk7	Tue wk8	Wed wk8
S-08	Tue wk7	Thu wk7	Wed wk8	Thu wk8
S-09	Thu wk7	Fri wk7	Thu wk8	Fri wk8
S-15 (pulled early)	Mon wk8	Tue wk8	Fri wk8	— not done
S-16 (pulled early)	Tue wk8	Wed wk8	Fri wk8	— not done

Deliverable: one page — board with limits, computed table, bottleneck and two proposals, the three-sentence argument. Two minutes to present: one assumption, one trade-off, one open question.

5

Homework · due before Session 7 (Tuesday 29 September)



Acceptance criteria

— Given/When/Then for the five highest-ordered stories not worked here. Each demonstrable at the post without explaining what to click.



Kanban write-up

— the one page from today's exercise, cleaned up, in the repo as

`s06-kanban.md`.



Reading

— Sommerville Ch. 3, §3.3–3.4.

Beck, *Extreme Programming*

Explained, ch. 1–3 (optional, short).



Assessment link. The backlog is the evidence base for Session 7's elicitation: every story is a hypothesis about what someone needs, and elicitation is how you find out which are wrong.

6

One-page recap

FIG. 6 · FIVE IDEAS · ONE RULE EACH · WHAT IT GAVE SANTE-BASSAM



If you remember one sentence: **agile is what a team does when it can afford to be wrong cheaply — and the line above which it cannot.**

Key terms

cost of feedback

Agile Manifesto

Scrum

Product Owner

sprint goal

Definition of Done

increment

velocity

XP

test-driven development

continuous integration

user story

INVEST

acceptance criteria

Kanban

WIP limit

cycle time

throughput

Little's Law

Readings and references

1. Sommerville, I. *Software Engineering*, 10th ed. — Chapter 3.
2. Beck, K. et al. *Manifesto for Agile Software Development*, 2001 — including the twelve principles.
3. Schwaber, K. & Sutherland, J. *The Scrum Guide*, 2020.
4. Beck, K. *Extreme Programming Explained: Embrace Change*, 2nd ed., 2004.
5. Anderson, D. *Kanban: Successful Evolutionary Change for Your Technology Business*, 2010.
6. Pressman & Maxim, Chapter 3.

International University of Grand-Bassam · School of Science and Engineering · **CSC 4305**, Fall 2026

Next · Session 07 — Requirements Engineering I: Why Users Cannot Tell You, and Who to Ask